

App Development Project Report

Placement Portal Application (PPA) — V2

1. Student Details

Field	Details
Name	Koustav Das
Roll Number	23f1003155
Email	23f1003155@ds.study.iitm.ac.in
About Me	I am a Data Science student with an academic background in Applied Mathematics, currently pursuing the BS in Data Science and Applications.

2. Project Details

Project Title:

Placement Portal Application (PPA) V2

Problem Statement:

Institutes require an efficient system to manage campus recruitment activities involving companies and students. Manual coordination through spreadsheets and emails makes it difficult to manage company approvals, placement drive creation, student registrations, and application tracking. This project builds a role-based Placement Portal web application that allows an Admin (Institute), Companies, and Students to interact with the system according to clearly defined permissions for each role.

Approach:

The backend is built with Flask, using separate route files (blueprints) for Admin, Company, Student, and Auth logic. Data is stored in SQLite, with Flask-SQLAlchemy handling database models and table creation — no manual database setup. Redis is used for two purposes: caching frequently accessed data, and acting as the broker for Celery, which runs three background jobs — an on-demand CSV export, a daily reminder, and a monthly report — with the last two scheduled through Celery Beat. The frontend is built with Vue 3, loaded directly via CDN with no build step, using Vue Router to navigate between role-based dashboards, and Bootstrap for styling. A single Jinja2 template serves as the entry point for the whole app.

3. AI/LLM Declaration

I took around 20% of help for styling, reactive frontend, routing, backend jobs and other from this AI/LLM is used only for debugging, understanding concept, removing error and organizing the code in a structured manner.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Backend framework for building the API
Flask-SQLAlchemy	ORM to create and manage the SQLite database
SQLite	Database for storing all application data
Flask-Caching	Connects Flask to Redis for caching
Redis	Caching backend and Celery message broker / result backend
Celery	Asynchronous task queue for background and scheduled jobs
Celery Beat	Scheduler for the daily reminder and monthly report jobs
Flask-Mail	Sending reminder and report emails (tested locally via MailHog)
Werkzeug	Handles password hashing
Vue.js 3 (CDN)	Builds the frontend, no build step needed

Vue Router	Handles navigation between role-based dashboards
Bootstrap 5	Used for styling and layout
Jinja2	Serves the single HTML entry point for the app

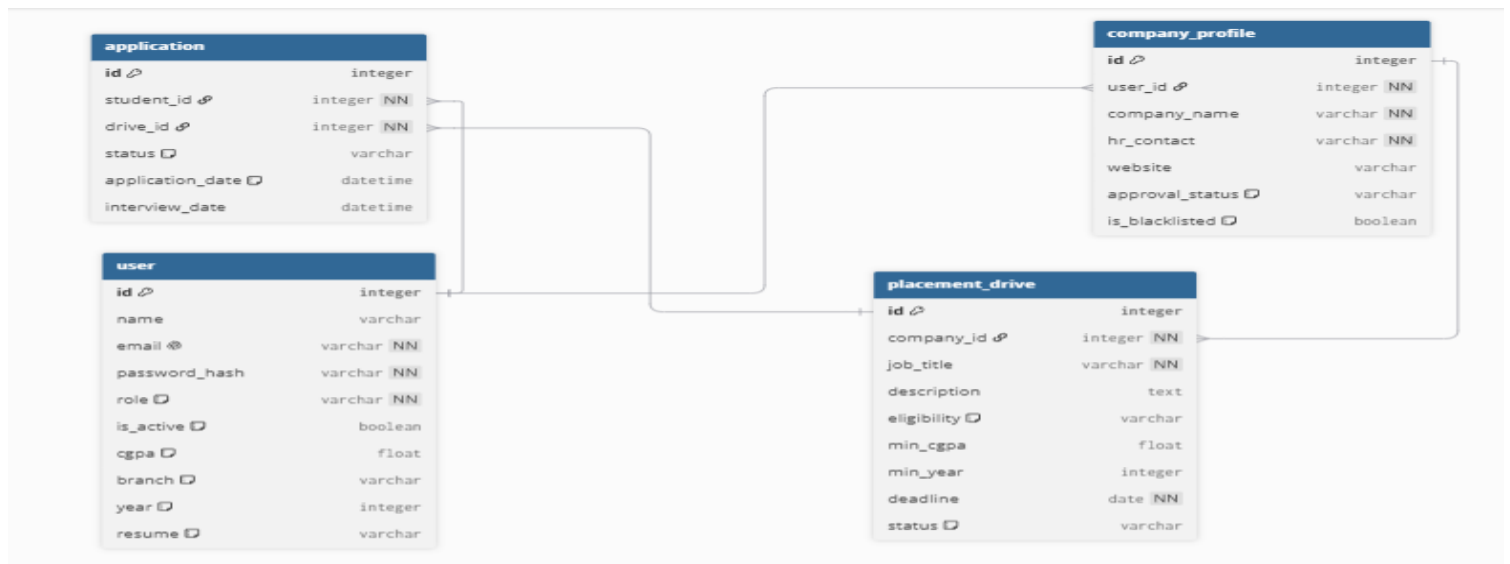
5. Database Schema

Tables:

- User — unified table for all three roles (id, name, email, password_hash, role, is_active, cgpa, branch, year, resume). Role-specific fields are NULL for roles that do not use them.
- CompanyProfile — company details linked to a User account (id, user_id [FK], company_name, hr_contact, website, approval_status, is_blacklisted).
- PlacementDrive — recruitment drives created by a company (id, company_id [FK], job_title, description, eligibility, min_cgpa, min_year, deadline, status).
- Application — a student's application to a drive (id, student_id [FK], drive_id [FK], status, application_date, interview_date).

Relationships:

- One-to-Many: User (company role) → CompanyProfile (restricted to one profile per user through application-level logic, not a database constraint)
- One-to-Many: CompanyProfile → PlacementDrive
- One-to-Many: User (student role) → Application
- One-to-Many: PlacementDrive → Application



6. API Resource Endpoints

Auth Blueprint

Endpoint	Method	Description
/auth/register	POST	Register a new student or company account (admin role blocked)
/auth/login	POST	Authenticate user and create a session
/auth/logout	GET	Clear the current session

Admin Blueprint

Endpoint	Method	Description
/admin/dashboard	GET	Overview counts: students, companies, drives, applications
/admin/companies	GET	List all company profiles
/admin/approve_company/<id>	POST	Approve a company profile
/admin/reject_company/<id>	POST	Reject a company profile
/admin/blacklist_company/<id>	POST	Blacklist a company
/admin/search_companies	GET	Search companies by name
/admin/drives	GET	List all placement drives
/admin/approve_drive/<id>	POST	Approve a placement drive (invalidates student cache)
/admin/reject_drive/<id>	POST	Reject a placement drive
/admin/search_drives	GET	Search drives by job title
/admin/students	GET	List all students
/admin/search_students	GET	Search students by name
/admin/deactivate_user/<id>	POST	Deactivate a student or company account
/admin/activate_user/<id>	POST	Reactivate a student or company account
/admin/applications	GET	View all applications with student and drive details
/admin/statistics	GET	Placement statistics: applied / shortlisted / selected / rejected counts

Company Blueprint

Endpoint	Method	Description
/company/create_profile	POST	Create the company profile (pending admin approval)
/company/profile	GET	View own company profile
/company/create_drive	POST	Create a placement drive (only if approved, not blacklisted)
/company/my_drives	GET	List own drives with applicant counts
/company/close_drive/<id>	POST	Close a placement drive
/company/applications/<drive_id>	GET	View applicants for a specific drive (ownership-checked)
/company/update_application/<id>	POST	Update an application's status
/company/schedule_interview/<id>	POST	Schedule an interview date for an applicant

Student Blueprint

Endpoint	Method	Description
/student/profile	GET	View own profile
/student/update_profile	POST	Update CGPA, branch, year
/student/upload_resume	POST	Upload resume PDF
/student/drives	GET	List approved drives (Redis-cached, 60s expiry)
/student/search_drives	GET	Search drives by job title
/student/search_companies	GET	Search approved companies by name
/student/apply	POST	Apply to a drive (eligibility + duplicate checks)
/student/my_applications	GET	View own applications with status
/student/placement_history	GET	View Selected/Rejected applications only
/student/export	GET	Trigger asynchronous CSV export (Celery)
/student/export_status/<task_id>	GET	Poll Celery task status
/student/download_csv/<filename>	GET	Download the generated CSV file

7. Architecture and Features

Architecture Overview:

- app.py — Flask application factory, configuration, blueprint registration, DB/cache initialisation, and programmatic admin seeding

- models.py — SQLAlchemy models: User, CompanyProfile, PlacementDrive, Application
- extensions.py — shared Cache object, isolated to avoid circular imports
- celery_worker.py — Celery application instance (Redis broker and result backend)
- tasks.py — three Celery tasks: CSV export, daily deadline reminder, monthly placement report
- beat_schedule.py — Celery Beat schedule configuration (Asia/Kolkata timezone)
- routes/ — Flask Blueprints: auth.py, admin.py, company.py, student.py
- static/ — Vue.js frontend: api.js (fetch wrapper), auth.js, admin.js, company.js, student.js, app.js (router + root app)
- templates/index.html — single Jinja2 entry point serving the Vue single-page application

Implemented Features:

- Session-based authentication with role-based access control (Admin / Company / Student)
- Admin: dashboard overview, company/drive approval and rejection, blacklisting, search, student/company activation control, full application visibility, placement statistics
- Company: profile registration gated on admin approval, drive creation with future-deadline validation, applicant management, status updates, interview scheduling
- Student: profile management, resume upload, eligibility-checked drive applications (CGPA, branch, year), duplicate-application prevention, placement history, application tracking
- Asynchronous, user-triggered CSV export of application history via Celery, with task-status polling and a download endpoint
- Scheduled daily deadline-reminder emails and a scheduled monthly HTML placement-activity report, both via Celery Beat
- Redis-backed caching of the student drives listing with a 60-second expiry and explicit cache invalidation on drive approval
- Responsive Bootstrap-based UI with role-specific tabbed dashboards built in Vue 3

Additional Notes:

- Email sending tested locally using MailHog
- Timestamps are generated in IST (Asia/Kolkata)
- On Windows, Celery Beat and the Celery worker are run as two separate processes, since Windows does not support the combined worker+beat mode.

8. Video Presentation

Drive Link: <https://drive.google.com/file/d/1-UfrbPKsTnJ1Oif1a0qrlUXSiiVGDgl/view?usp=sharing>